

Προγραμματιστική Άσκηση: Η γλώσσα προγραμματισμού cru

Κατά την διαδικασία δημιουργίας του λεκτικού αναλυτή διαβάζεται το `cry` αρχείο λεκτική μονάδα- λεκτική μονάδα. Κάθε λεκτική μονάδα που εντοπίζεται, τοποθετείται στην λίστα `myList` η οποία στο τέλος τη λεκτικής ανάλυσης θα περιέχει όλες τις λεκτικές μονάδες του αρχείου. Επιπλέον τοποθετείται στην λίστα `family` το είδος της λεκτικής μονάδας. Τα είδη που μπορεί να είναι μία μονάδα είναι `taken`, `celebrant`, `rel_op`, `identifier`, `constant`, `add_op`, `mul_op` και `comment`. `Taken` θεωρούνται οι λεκτικές μονάδες που είναι δεσμευμένες από την γλώσσα και δεν μπορούν να χρησιμοποιηθούν ως μεταβλητές, πχ: `#def`, `# #int` κλπ. `Celebrant` είναι οι διαχωριστές καθώς και τα σύμβολα ομαδοποίησης, `rel_op` είναι τα σύμβολα συσχέτισης, `add_op` είναι τα σύμβολα των αριθμητικών πράξεων πρόσθεσης, αφαίρεσης και `mul_op` είναι τα σύμβολα των αριθμητικών πράξεων πολλαπλασιασμού, διαίρεσης, υπόλοιπου. Τέλος `identifier` είναι συμβολοσειρές που αποτελούνται από γράμματα και ψηφία, `constant` είναι οι αριθμοί και `comment` είναι τα σχόλια.

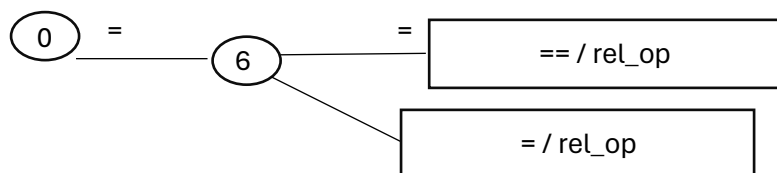
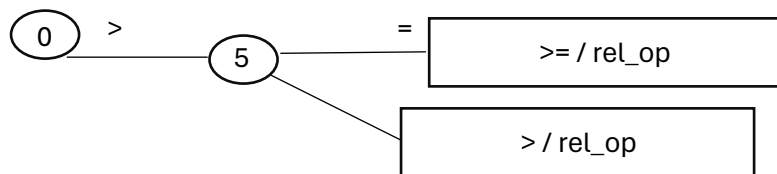
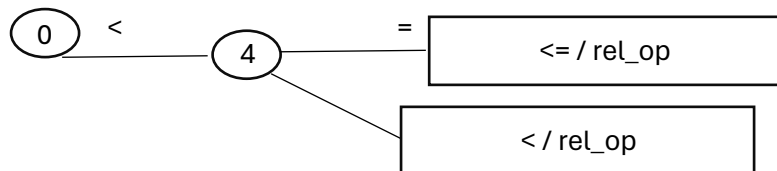
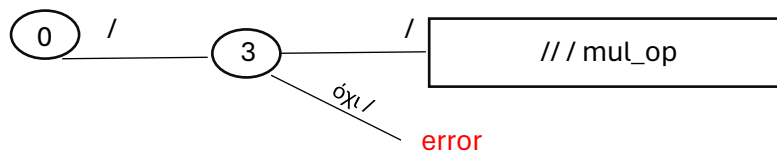
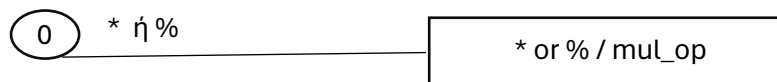
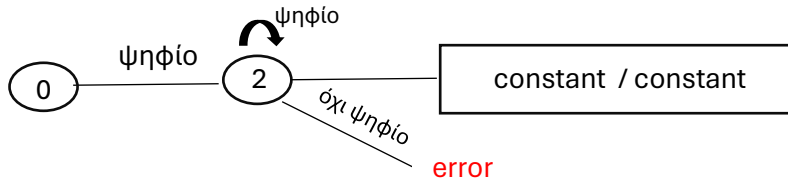
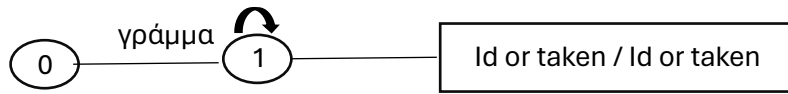
Αρχικά, δηλώνονται οι καθολικές μεταβλητές: `state`, `line_counter`, `word` και `checkifmissed`. Η `state` κρατάει την τρέχουσα κατάσταση του αναλυτή, η `line_counter` μετράει τον αριθμό των γραμμών στον πηγαίο κώδικα, η `word` συσσωρεύει χαρακτήρες για να σχηματίσει μια λέξη ή ένα `token`, και η `checkifmissed` είναι μια σημαία που καθορίζει αν ο τελευταίος χαρακτήρας πρέπει να επανεξεταστεί στην τρέχουσα κατάσταση. Η λεκτική ανάλυση σταματά είτε όταν το `state` γίνει ίσο με `OK`, δηλαδή όταν φτάσει στο τέλος του αρχείου `cry`, είτε όταν το `state` γίνει `-1`, δηλαδή όταν εντοπίζεται λάθος. Ανάλογα με την τρέχουσα κατάσταση και την λεκτική μονάδα που διαβάστηκε, ο αναλυτής μεταβαίνει μεταξύ καταστάσεων και εκτελεί ενέργειες. Για παράδειγμα, αν η `state` είναι `0` και η λεκτική μονάδα είναι αλφαβητικός χαρακτήρας, μεταβαίνει στην κατάσταση `1` για να αρχίσει να σχηματίζει μια λέξη. Αν η `state` είναι `1` και η λεκτική μονάδα δεν είναι αλφαβητικός ή αριθμητικός χαρακτήρας, ο αναλυτής ολοκληρώνει τη λέξη, ελέγχει αν είναι λέξη-κλειδί (στη λίστα `taken`), την προσθέτει στη `myList` και επαναφέρει την `state` στο `0`.

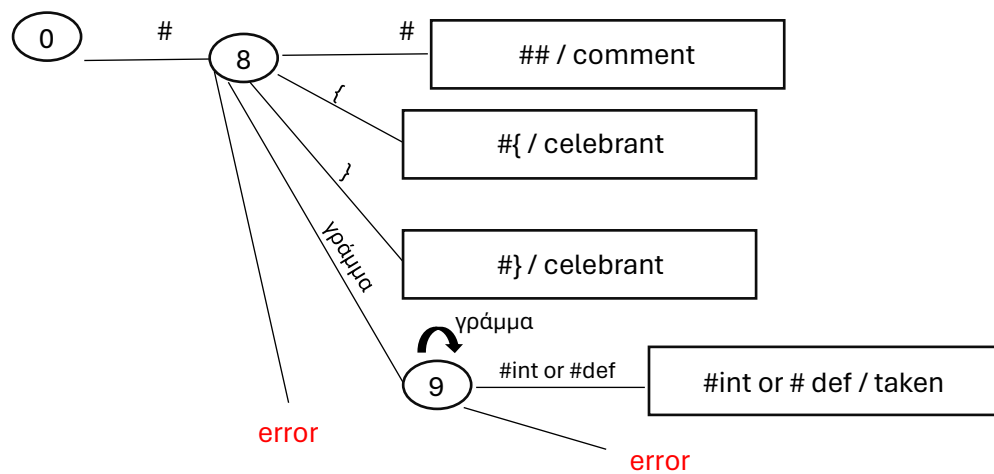
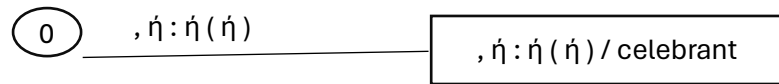
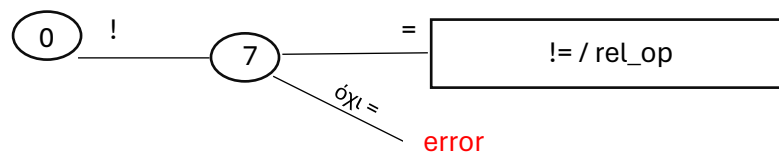
Στο τέλος της λεκτικής ανάλυσης η λίστα `myList` περιέχει όλες τις λεκτικές μονάδες του `.cry` αρχείου, η λίστα `family` περιέχει τον τύπο της κάθε λεκτικής μονάδας και η λίστα `line_number` περιέχει την γραμμή στην οποία βρίσκεται η κάθε μονάδα.

Παρακάτω ακολουθεί σχήμα επεξήγησης των καταστάσεων:

state =  token/family = 

γράμμα ή ψηφίο





Συντακτικός Αναλυτής

Η κύρια δουλειά του συντακτικού αναλυτή είναι να ελέγχει τη συντακτική δομή μιας γλώσσας προγραμματισμού σύμφωνα με τους κανόνες γραμματικής της γλώσσας.

Οι κανόνες της γραμματικής είναι :

program : defcomment hashtagint defdef hashtagdef;

hashtagint: ('#int' ID (',' ID)*)*;

defglobal : ('global' ID (',' ID)*)*;

defdef : 'def' ID '(' actual_par_list ')' ':'

defcomment

'#{'

defcomment

```

        hashtagint
    defcomment
    defdef
    defcomment
    defglobal
    defcomment
    block

'##';

hashtagdef  : '#def' 'main' defcomment hashtagint defcomment block
defcomment

code_block  : defstatement (defstatement)* ;

defstatement : defassignment| defprint| defreturn | defif | defwhile ;

defassignment: ID '=' ( expression | 'int' '(' 'input' '(' ' '));

defprint    : 'print' '(' expression ')';

defreturn   : 'return' (expression | '(' expression ')');

defif       : 'if' condition ':' ( '#{ ' block #} | defstatement) (defelif | ) (defelse | );

defelif     : ( 'elif' condition ':' ( '#{ ' block #} | defstatement))*;

defelse     : 'else' ':' ( '#{ ' block #} | defstatement));

defwhile    : 'while' condition ':' ( '#{ ' block #} | defstatement));

expression  : optional_sign term ( add_oper term )*;

term        : factor ( mul_oper factor )*;

factor      : constant | '(' expression ')' | ID idtail ;

idtail      : '(' funcparam ')' | ;

actual_par_list: expression ( ';' expression )*;

funcparam   : expression ( ';' expression )*;

defcomment  : ## (...)## ;

condition   : bool_term ( 'or' bool_term )*;

bool_term    : bool_factor ( 'and' bool_factor )*;

bool_factor  : expression rel_op expression | 'not' expression rel_op
expression;

```

```
optional_sign : add_oper | ;  
add_oper      : '+' | '-' ;  
mul_oper      : '*' | '/' | '%' ;  
rel_op        : '>' | '<' | '>=' | '<=' | '==' | '!=' ;
```

Γενικότερα η γραμματική είναι όσο πιο κοντά γινόταν στην γραμματική που δόθηκε από τον διδάσκοντα. Έγινε προσθήκη της `defcomment` η οποία έχει ενημερωτικό ρόλο σε περίπτωση που η απόσταση των δύο `## ##` είναι μεγάλη. Δουλειά της είναι να ενημερώνει τον χρήστη ότι έχει ανοίξει ένα σχόλιο αλλά πιθανότατα δεν το έχει κλείσει.

Επίσης ο λόγος που η `funcparam` και η `actual_par_list` κάνουν την ίδια δουλειά, είναι για να κάνει πιο εύκολη την διαδικασία κατά την διάρκεια παραγωγής του ενδιαμέσου κώδικα και του πίνακα συμβόλων. Η `funcparam` πρακτικά είναι υπεύθυνη για την ανάλυση των παραμέτρων μιας μεθόδου όταν καλείται σε μία άλλη μέθοδο, ενώ η `actual_par_list` είναι υπεύθυνη για την ανάλυση των παραμέτρων της συνάρτησης όταν γίνεται η συντακτική ανάλυση της ίδιας της συνάρτησης.

Σε περιπτώσεις που κατά την συντακτική ανάλυση δεν πληρούνται οι προϋποθέσεις της γραμματικής, ο χρήστης λαμβάνει μήνυμα λάθους, υποδεικνύοντας, όπου είναι δυνατόν, με ακρίβεια τον τύπο του λάθους που εντοπίστηκε και σε ποια γραμμή του κώδικα. Η συντακτική ανάλυση δεν επιστρέφει κάποιο αρχείο στον χρήστη.

Ενδιάμεσος Κώδικας

Για την παραγωγή του ενδιαμέσου κώδικα υλοποιήθηκαν κάποιες μέθοδοι ώστε να γίνεται πιο γρήγορα και εύκολα η όλη διαδικασία. Η πρώτη μέθοδος που δημιουργήθηκε είναι η `genQuad`. Η συγκεκριμένη μέθοδος δημιουργεί μία νέα τετράδα και την προσθέτει στην λίστα από τετράδες. Η τετράδα που δημιουργείται είναι της μορφής `quad = [label, operator, operand1, operand2, operand3]`. Επιπλέον αυξάνει το `label` κατά 1, ώστε να το λάβει η επόμενη τετράδα ενημερωμένο. Η επόμενη μέθοδος είναι η `nextQuad` η οποία απλά επιστρέφει το `label` της επόμενης τετράδας. Επιπλέον η `emptyList` δημιουργεί μία κενή λίστα, ενώ η `makeList(x)` δημιουργεί μια κενή λίστα, προσθέτει μέσα της το `x` και την επιστρέφει. Η `backpatch(list, label)` προσθέτει το `label` στο τελευταίο στοιχείο κάθε υπολίστας της `list`. Σκοπός της `backpatch` είναι να ενημερώνει τις τετράδες που θα οδηγούν σε άλλη τετράδα (π.χ. `jump, _, _, 4`). Τέλος η `newTemp` δημιουργεί μία καινούρια

προσωρινή μεταβλητή και την επιστρέφει και η saveEndiamesos δημιουργεί το αρχείο endiamesos.int, στο οποίο γράφει όλα τα δεδομένα της λίστας quads που περιέχει όλες τις τετράδες. Κάποιες από τις εντολές που χρησιμοποιήθηκαν και αξίζουν να σχολιαστούν είναι :

Στην περίπτωση της εισόδου δεδομένων, στην defassignemnt γίνεται η εξής διαδικασία `w = newTemp() genQuad("inp","_","_",w)` `genQuad(":=,w,"_","_",name)`. Δηλαδή δημιουργείται μία νέα προσωρινή μεταβλητή και μία νέα τετράδα στην οποία η w γίνεται τύπου `inp`. Τέλος δημιουργείται μια έξτρα τετράδα στην οποία η μεταβλητή γίνεται ίση με την προσωρινή μεταβλητή.

Μία ακόμη αλλαγή που έγινε είναι στην περίπτωση της `return`. Στην `return` προστέθηκαν οι εντολές `temp = makeList(nextQuad()) returnlist.append(temp)` `genQuad("jump","_","_",") flagreturn=1`. Ο σκοπός αυτών είναι να προστίθεται μια νέα τετράδα `jump` κάτω από κάθε τετράδα η οποία οδηγεί στο τέλος της μεθόδου. Στο τέλος της `defdef` έχουν προστεθεί οι εντολές `backPatch(returnlist,nextQuad()) returnlist.clear()` οι οποίες πρακτικά παίρνουν τις τετράδες με `jump` που ακολουθούν τις `return` της μεθόδου, και στο τελευταίο στοιχείο τους βάζουν την τετράδα που οδηγεί στο τέλος της μεθόδου. Για να μην υπάρχει διπλή `jump` στην περίπτωση που μέσα σε `if` και `elif` υπάρχει `return`, στις `defif` και `defelif` ελέγχουμε αν η σημαία `flagreturn` είναι ίση με 0 ή 1. Αν είναι ίση με 0 τότε σημαίνει ότι δεν υπήρξε `return` στον κώδικα της `if` ή της `elif` και προστίθενται το δικό τους `jump`, ενώ στην περίπτωση που είναι 1 τότε αυτό το βήμα παραλείπεται.

Ένα ακόμα σχόλιο για την παραγωγή του ενδιάμεσου κώδικα στην `if` είναι ότι στην περίπτωση που δεν υπάρχει `else` μετά από `if` τότε, ελέγχοντας το αυτό με την σημαία `flag2`, γίνεται `backpatch` στην λίστα της `if` όταν ολοκληρωθεί όλος ο κώδικας που περιέχει η `if`. Έτσι επιτυγχάνεται να μην υπάρχουν `jump` με κενό το τελευταίο στοιχείο τους.

Πίνακας Συμβόλων

Για την παραγωγή του πίνακα συμβόλων δημιουργήθηκαν οι: `createScope`, η οποία δημιουργεί ένα νέο `scope`, το οποίο περιέχει μια αρχική κενή λίστα από `enitities` και το `nesting level`. Προσθέτει το `scope` στην λίστα `scopes`, που περιέχει όλα τα `scopes`, αυξάνει το `nesting level` και ενημερώνει το `current_scope` με το τελευταίο στοιχείο της λίστας `scopes`. Άλλη βοηθητική μέθοδος που δημιουργήθηκε είναι η `deleteScope` η οποία διαγράφει το τελευταίο `scope` της `scopes`, μειώνει το `nesting level` κατά και ενημερώνει κατάλληλα το `current_scope`. Επιπλέον σε αυτήν την μέθοδο ενημερώνεται και η `scope_temp` η οποία θα φανεί χρήσιμη κατά την παραγωγή του τελικού κώδικα. Επιπλέον δημιουργήθηκαν οι `insertEntity...` οι οποίες είναι υπεύθυνες για την δημιουργία

entity για μεταβλητές, παραμέτρους, συναρτήσεις, προσωρινές μεταβλητές και σταθερές. Επίσης η searchEntity ελέγχει σε όλο τον πίνακα συμβόλων αν υπάρχει κάποιο στοιχείο. Αν υπάρχει επιστρέφει την πρώτη εμφάνιση του αλλιώς επιστρέφει -1. Οι updateOffset και findLastOffset είναι υπεύθυνες για την επιστροφή του ενημερωμένου offset όταν προστίθεται ένα νέο entity σε ένα score και για την απλή επιστροφή του offset του τελευταίου entity αντίστοιχα. Τέλος η saveSymbol δημιουργεί το αρχείο symbol.sym που περιέχει όλα τα scores από την αρχή μέχρι το τέλος της διαδικασίας. Για να επιτευχθεί αυτό κάθε φορά που πάει να διαγραφεί, προστίθεται ένα deepcopy του στην λίστα scores_tobeprinted. Έτσι έχουμε όλη την εικόνα των scores από την αρχή μέχρι το τέλος.

Η δημιουργία των scores γίνεται κάθε φορά που μεταγλωττίζεται μία νέα συνάρτηση και στην αρχή του προγράμματος. Το score που δημιουργείται στην αρχή της μεταγλώττισης είναι και αυτό που θα διαγραφεί τελευταίο, καθώς κάθε καινούριο που θα δημιουργείται θα είναι ένα πιο βαθιά από αυτό. Στο τέλος της μεταγλώττισης το score 0, λίγο πριν διαγραφεί και αυτό, θα περιλαμβάνει όλες τις global μεταβλητές και τις συναρτήσεις, εκτός από τις φωλιασμένες. Γενικότερα ο πίνακας συμβόλων είναι χρήσιμος για τον εντοπισμό των μεταβλητών και για τον διαχωρισμό των global και μη μεταβλητών.

Τελικός Κώδικας

Ο τελικός κώδικας αποτελεί το τελευταίο στάδιο της διαδικασίας. Σε αυτό το στάδιο όλα όσα έχουν γίνει έως αυτό το σημείο χρησιμοποιούνται ώστε να γραφτούν σε γλώσσα assembly. Σε όλη την διαδικασία παραγωγής του τελικού κώδικα παίζει καθοριστικό ρόλο η searchEntity, καθώς ψάχνει για την μεταβλητή που τις δίνεται και ελέγχει για αρχή αν υπάρχει. Αν δεν υπάρχει επιστρέφεται μήνυμα λάθους και σταματά η μεταγλώττιση. Στην περίπτωση που υπάρχει, όμως, μας δίνει την δυνατότητα να ξεχωρίσουμε αν είναι global ή όχι.

Για την παραγωγή του τελικού κώδικα δημιουργήθηκαν οι βοηθητικές μέθοδοι loadnr και storern. Σε αυτές γίνεται η παραγωγή των εντολών lw και sw αντίστοιχα. Είναι σημαντικό ότι σε αυτές τις μεθόδους ελέγχουμε από τον πίνακα συμβόλων αν οι μεταβλητές τις οποίες διαχειρίζονται είναι global ή όχι. Αν είναι τότε γίνεται χρήση του gp(global pointer) αλλιώς του sp(stack pointer). Είναι καίριο να ελέγχουμε όμως αν η μεταβλητή την οποία ψάχνουμε είναι στην μέθοδο η οποία μεταγλωττίζεται, αλλιώς θα πρέπει να βρούμε την θέση της από το εγγράφημα δραστηριοποίησης. Αυτό το επιτυγχάνει η gnlncode. Επίσης κατά την παραγωγή του τελικού η πρώτη εντολή που γράφεται είναι αυτή που στέλνει στην main καθώς θέλουμε η πρώτη μέθοδος που θα υλοποιηθεί να είναι η κύρια μέθοδος, και οι υπόλοιπες θα τρέξουν μόνο αν καλεστούν από αυτήν. Στην main αντί για move

gr,sp , χρησιμοποιήθηκε η addi gr,gr,framelength καθώς η προηγούμενη δεν λειτουργούσε όπως αναμενόταν.

Ο τελικός κώδικας που παράγεται δεν καταφέρνει να τα βγάλει πέρα με τις φωλιασμένες συναρτήσεις. Παρόλο που ακολουθήθηκαν όλες οι οδηγίες, τα προγράμματα που περιέχουν φωλιασμένες συναρτήσεις δεν λειτουργούν σωστά. Για αυτόν τον λόγο παραδίδω και test τα οποία περιέχουν πολλές λειτουργίες, εκτός από φωλιασμένες συναρτήσεις.

Ο τελικός κώδικας επιστρέφει το αρχείο final.asm, το οποίο περιέχει όλες τις εντολές assembly.